

MONITOR Z · L07

Ser un STC de verdad

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L07 — Ser un STC de verdad

Objetivo:

- que el monitor arranque con `S ZTKMON`;
- que atienda órdenes del operador (`F ZTKMON, ESTADO`);
- que pare **ordenado** con `P ZTKMON`;
- que, si algo falla, deje dicho qué pasó.

 **Requiere de sistemas:** el miembro de la PROC en la PROCLIB y el perfil en la clase STARTED. La parte de órdenes del operador **se puede practicar antes** con un job batch (ver la práctica).

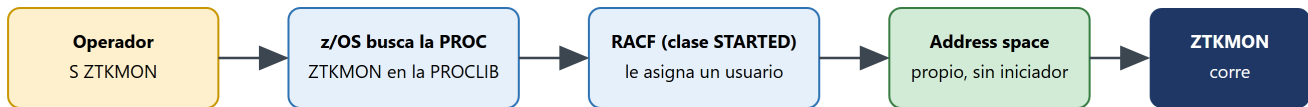
1. Qué cambia de job a STC

	Job batch largo	STC
Arranca con	<code>SUBMIT</code>	<code>S ZTKMON</code> (o el arranque del sistema)
JCL	El miembro que sometés	Una PROC en la PROCLIB
Iniciador	Ocupa uno todo el tiempo	No usa iniciador batch: tiene su propio address space
Usuario	El tuyo	El que asigna RACF por la clase STARTED
Se para con	Cancelar (o que termine solo)	<code>P ZTKMON</code> : ordenado

El programa es casi el mismo. Lo que cambia es cómo se arranca y que **tiene que atender al operador.**

2. Cómo arranca

1. El operador escribe `S ZTKMON`.
2. z/OS busca la PROC `ZTKMON` en la PROCLIB.
3. RACF le asigna el usuario según el perfil STARTED.
4. Se crea el address space y se ejecuta el programa.



Lo que hay que pedirle a sistemas: el miembro en la PROCLIB, el perfil en la clase STARTED, y (si usa ENF 70) que la librería sea APF.



Analogía: el empleado de planta

Para que un empleado de planta empiece a trabajar hace falta su **contrato** (la PROC en la PROCLIB), su **credencial** (el usuario de la clase STARTED) y su **oficina** (el address space). Después llega solo cada mañana cuando lo llaman (S ZTKMON).

Fuente: z/OS MVS System Commands (START) · z/OS Security Server RACF (clase STARTED) · criterio propio para el diagrama

Lámina L07-01_como_arranca_un_stc

3. Recibir órdenes: EXTRACT, COM, CIB y QEDIT

#	Paso	Detalle
1	EXTRACT answer, FIELDS=COMM	Deja en <code>answer</code> la dirección del área COM (mapeo <code>IEZCOM</code>)
2	COMECBPT	Dirección del ECB que el sistema postea cuando llega un comando → va a la <code>ECBLIST</code> del bucle (L02)
3	COMCIBPT	Dirección del primer CIB (mapeo <code>IEZCIB</code>): el comando en sí
4	CIBVERB	Qué fue: START, MODIFY (F) o STOP (P)
5	CIBDATLN / CIBDATA	Largo y texto del MODIFY, por ejemplo <code>ESTADO</code>
6	QEDIT ORIGIN=..., BLOCK=...	Liberar el CIB atendido; si no, se acumulan
7	QEDIT ORIGIN=..., CIBCTR=n	Permitir hasta <i>n</i> MODIFY encolados

Detalles a confirmar en la práctica (se toman de los mapeos IEZCOM/IEZCIB de tu SYS1.MACLIB):

- Los valores exactos de `CIBVERB`.
- Si al arrancar con parámetros queda un **CIB de START** encolado. Si queda, hay que leerlo (trae los parámetros) y liberarlo con QEDIT antes de entrar

al bucle.

Lo que haría ZTKMON con cada orden (propuesta):

Orden	Acción
F ZTKMON, ESTADO	WTO con vueltas, eventos, clientes conectados y CPU
P ZTKMON	Cerrar el socket, mandar +FIN, cerrar SYSPRINT y terminar con RC 0
Cualquier otro texto	WTO "orden desconocida", y nada más

 Fuente: z/OS MVS Programming: Assembler Services Guide, "Communicating with the system operator" (EXTRACT, QEDIT, CIB).

Recibir F y P del operador: EXTRACT, CIB y QEDIT

L07 · Ser un STC de verdad

#	Paso	Detalle
1	EXTRACT ... FIELDS=COMM	Devuelve la dirección del área COM (mapeo IEZCOM)
2	En el COM: COMECBPT	Dirección del ECB que el sistema postea cuando llega un comando → va a la ECBLIST del bucle (L02)
3	En el COM: COMCIBPT	Dirección del primer CIB (mapeo IEZCIB): el comando en sí
4	Leer CIBVERB	Qué fue: START , MODIFY (F ZTKMON, texto) o STOP (P ZTKMON)
5	Leer CIBDATA / CIBDATLN	El texto del MODIFY, por ejemplo ESTADO
6	QEDIT ORIGIN=..., BLOCK=...	Liberar el CIB ya atendido (si no, se acumulan)
7	QEDIT ORIGIN=..., CIBCTR=n	Permitir hasta <i>n</i> MODIFY encolados

Comando	Qué haría ZTKMON (propuesta)
F ZTKMON, ESTADO	WTO con vueltas, eventos, clientes conectados y CPU
P ZTKMON	Cerrar el socket, mandar +FIN, cerrar SYSPRINT y terminar con RC 0

 **Analogía: el buzón de órdenes de la oficina**
Cuando el gerente deja una orden, se levanta la banderita (el ECB del COM). El empleado abre el buzón, lee el papel (el CIB), hace lo que dice y **tira el papel** (QEDIT); si no lo tira, el buzón se llena.

Los valores exactos de CIBVERB se toman del mapeo IEZCIB al escribir la lección.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Communicating with the system operator": EXTRACT, QEDIT, CIB)

Lámina L07-02_comandos_del_operador

4. El bucle final

Con esto, la **ECBLIST** de L02 queda completa:

ECB	Lo postea	Acción
Timer	La salida de STIMERM	Una vuelta de SSI 80
COM (COMECBPT)	El sistema, al llegar F / P	Leer el CIB, actuar y liberarlo
Socket (L06)	La API, si se usa el modo con ECB	Leer el pedido de la PC
Evento (L08)	El listen exit de ENF 70	Mandar +EVT

5. Recuperación: ESTAEX

Sin recuperación, unabend mata el STC sin dejar rastro en el monitor, y el portal se entera sólo por COLECTOR caído.

Con **ESTAEX**:

- El sistema llama a **tu rutina de recuperación**, con el diagnóstico en el **SDWA**.
- La rutina puede **reintentar** en un punto seguro (*retry*) o dejar seguir elabend (*percolate*).
- **Para un monitor, la política sugerida** es registrar (WTO + SYSPRINT), cerrar el socket ordenado y **terminar**. No hay que ocultar el error.

 Fuente: *Assembler Services Guide*, "Providing recovery" (ESTAEX, SDWA, retry y percolate).

Sin recuperación	Con <code>ESTAE</code>	Qué ganamos
Elabend mata el STC; el socket queda abierto hasta que TCP/IP lo limpie	El sistema llama a tu rutina de recuperación con el diagnóstico (SDWA)	Escribir qué pasó (WTO + SYSPRINT)
El portal se entera por COLECTOR caído (180 s)	La rutina puede reintentar en un punto seguro (<i>retry</i>) o dejar que siga elabend (<i>percolate</i>)	Mandar <code>+ERR</code> o cerrar el socket ordenado
No hay rastro de la causa en el monitor	Para un monitor: registrar, cerrar limpio y terminar (no ocultar el error)	Que el operador vea un mensaje claro



Analogía: el paracaídas de reserva

ESTAE es el paracaídas de reserva: no evita que algo falle, pero permite **aterrizar ordenado** y dejar dicho qué pasó.

Reentrancia (repaso de L01): un STC de varias tareas debería tener sus datos de trabajo en memoria propia (`STORAGE OBTAIN`), no dentro de la CSECT.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Providing recovery": ESTAE, SDWA, retry y percolate)

Lámina L07-03_recuperacion_estae

6. Reentrancia y memoria propia

En L01 vimos que ZTKMONS v0 tiene sus datos **dentro de la CSECT**. Para el STC conviene:

- pedir el área de trabajo con `STORAGE OBTAIN` al arrancar (y `STORAGE RELEASE` al terminar);
- apoyar los datos con un `USING` sobre una DSECT;
- dejar la CSECT **sin modificar**.

Así, si en algún momento hay más de una tarea (por ejemplo, el exit de L08), nadie escribe sobre los datos de otro.

7. Preguntas de comprobación

1. ¿Qué pasa si nunca liberás los CIB con QEDIT y el operador manda veinte `F ZTKMON, ESTADO` ?
2. ¿Por qué el ECB del COM tiene que estar en la `ECBLIST` y no consultarse cada tanto?
3. ¿Qué diferencia hay entre *retry* y *percolate*? ¿Cuál conviene para un monitor y por qué?
4. ¿Qué tiene que pedirle a sistemas el STC que no pedía el job batch?

5. ¿Por qué una CSECT que se modifica a sí misma es un problema cuando hay más de una tarea?

Mis respuestas

8. Práctica P07: ZTKOIDO (sin pedir nada a sistemas)

Un **job batch** que:

1. Hace `EXTRACT ... FIELDS=COMM` y `QEDIT ... CIBCTR=5`.
2. Espera en una `ECBLIST` con el ECB del COM y un timer de 5 minutos.
3. Ante `F ZTK0ID0,ESTADO` contesta con un `WTO` (vueltas, hora de inicio).
4. Ante `P ZTK0ID0` termina ordenado, con RC 0.
5. Si pasan los 5 minutos sin orden, termina con RC 4.

A confirmar en DESA: que `F` y `P` lleguen también a un job batch que hizo `EXTRACT ... COMM`. Es lo esperable, pero lo prueba esta práctica.

Traé el JOBID, el SYSLOG o JESMSGLOG con tus F/P y sus respuestas, y el RC final.

Relacionado: *L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB* · *L06 - Sockets en z-OS* · siguiente: *L08 - Eventos de verdad - ENF 70*.

Fuente: *Assembler Services Guide* (EXTRACT, QEDIT, ESTAEX, STORAGE) · *z/OS MVS System Commands* (START, MODIFY, STOP) · *z/OS Security Server RACF* (clase STARTED).

Notas